

Database Assignments — Complete Reference Guide

SQL · PL/SQL · Views & Indexes · Cassandra CQL · MongoDB — Topic-wise Solved Patterns

How to use this guide

Every assignment you uploaded (SQL basics, joins, functions, constraints, PL/SQL procedures/triggers, views/indexes, Cassandra, MongoDB) is really the **same handful of patterns repeated on different tables**. This guide gives you one clean, correct, working example of every pattern, using the standard EMP / DEPT / SALGRADE tables (or the assignment's own tables where relevant), so you can swap column/table names and answer *any* similar question yourself.

Standard reference tables used below:

```
-- EMP(empno, ename, job, mgr, hiredate, sal, comm, deptno)
-- DEPT(deptno, dname, loc)
-- SALGRADE(grade, losal, hisal)
```

PART A — CORE SQL (Assignments 1, 2, 3, 4, 5, SLA-1)

A1. Comparison, Logical, Range, List operators

```
-- sal > 2000 AND comm > 200
SELECT * FROM emp WHERE sal > 2000 AND comm > 200;

-- job = 'CLERK' OR sal > 2000
SELECT * FROM emp WHERE job = 'CLERK' OR sal > 2000;

-- value in a list
SELECT * FROM emp WHERE sal IN (1250, 1100, 2850);
SELECT * FROM emp WHERE sal NOT IN (1250, 1100, 2850);

-- range (inclusive)
SELECT * FROM emp WHERE sal BETWEEN 1250 AND 2850;
SELECT * FROM emp WHERE sal > 1250 AND sal < 2850; -- exclusive range

-- NULL checks (never use = NULL)
SELECT * FROM emp WHERE comm IS NULL;
SELECT * FROM emp WHERE sal IS NULL AND ename LIKE 'S%';
```

A2. Pattern matching with LIKE

% = any number of characters, _ = exactly one character.

```
SELECT * FROM emp WHERE ename LIKE '%AS'; -- ends with AS
SELECT * FROM emp WHERE job LIKE 'C%K'; -- starts C, ends K
SELECT * FROM emp WHERE job LIKE '__L%M_'; -- L at 3rd pos, M 3rd-last (adjust underscores to position)
SELECT * FROM emp WHERE job LIKE 'C_E%K'; -- starts C, E at 3rd pos, ends K
SELECT * FROM emp WHERE ename LIKE 'A%N'; -- starts A ends N
SELECT * FROM emp WHERE ename LIKE '%N%' AND ename LIKE 'A%S'; -- A..N..S pattern combine
SELECT * FROM emp WHERE ename LIKE '_R%'; -- R at 2nd-last -> use LIKE '%R_'
SELECT * FROM emp WHERE ename LIKE '%_%' ESCAPE '\'; -- literal underscore in name
SELECT * FROM emp WHERE LENGTH(job) = 5; -- job contains exactly 5 chars
SELECT * FROM emp WHERE ename LIKE 'A%' AND LENGTH(job) = 5;
```

Position-based rule of thumb: count underscores from the side you're anchoring — `'__L%'` means "any 2 chars, then L", `'%M__'` means "M, then any 2 chars, end of string".

A3. Column aliases, DISTINCT, CONCAT, string ops

```
SELECT first_name AS "WORKER_NAME" FROM worker;
SELECT UPPER(first_name) FROM worker;
SELECT DISTINCT department FROM worker;
SELECT SUBSTR(first_name,1,3) FROM worker; -- first 3 chars
SELECT INSTR(first_name,'a') FROM worker; -- position of 'a'
SELECT first_name || ' ' || department FROM worker; -- concat with space
SELECT LTRIM(department) FROM worker; -- trim left spaces
SELECT LENGTH(DISTINCT_DEPT) FROM (SELECT DISTINCT department AS distinct_dept FROM worker);
SELECT REPLACE(first_name,'a','A') FROM worker;
SELECT first_name || ' ' || last_name AS complete_name FROM worker;
SELECT * FROM worker ORDER BY first_name ASC;
SELECT * FROM worker ORDER BY first_name ASC, department DESC;
SELECT * FROM worker WHERE first_name IN ('Vipul','Satish');
SELECT * FROM worker WHERE first_name NOT IN ('Vipul','Satish');
SELECT * FROM worker WHERE department = 'Admin';
SELECT * FROM worker WHERE first_name LIKE '%a%';
SELECT * FROM worker WHERE first_name LIKE 'a%';
SELECT * FROM worker WHERE first_name LIKE '%h' AND LENGTH(first_name)=6;
SELECT * FROM worker WHERE salary BETWEEN 100000 AND 500000;
SELECT * FROM worker WHERE joining_date BETWEEN DATE'2014-02-01' AND DATE'2014-02-28';
SELECT COUNT(*) FROM worker WHERE department='Admin';
```

```

SELECT first_name FROM worker WHERE salary BETWEEN 50000 AND 100000;
SELECT department, COUNT(*) FROM worker GROUP BY department ORDER BY COUNT(*) DESC;

```

A4. Set / duplicate / positional tricks (Assignment 2, Q24-50)

```

-- workers who are also managers (self-lookup, generic pattern)
SELECT * FROM worker WHERE worker_id IN (SELECT DISTINCT manager_id FROM worker);

-- duplicate rows on some columns
SELECT first_name, last_name, COUNT(*) FROM worker
GROUP BY first_name, last_name HAVING COUNT(*) > 1;

-- odd / even rows (needs a row number)
SELECT * FROM (SELECT w.*, ROW_NUMBER() OVER (ORDER BY worker_id) rn FROM worker w)
WHERE MOD(rn,2)=1;      -- odd rows; use =0 for even

-- clone a table structure + data
CREATE TABLE worker_copy AS SELECT * FROM worker;

-- intersecting / difference records
SELECT * FROM table1 INTERSECT SELECT * FROM table2;
SELECT * FROM table1 MINUS SELECT * FROM table2;      -- rows in t1 not in t2 (Oracle); EXCEPT in SQL Server/
Postgres

SELECT SYSDATE FROM dual;                          -- current date & time (Oracle); NOW() in MySQL

SELECT * FROM worker FETCH FIRST 10 ROWS ONLY;      -- top-N (Oracle 12c+); LIMIT 10 in MySQL/Postgres

-- nth highest salary
SELECT * FROM (SELECT DISTINCT salary FROM worker ORDER BY salary DESC)
WHERE ROWNUM = 5;      -- Oracle: 5th highest (wrap ROWNUM logic)
-- Portable version:
SELECT salary FROM worker w1
WHERE 4 = (SELECT COUNT(DISTINCT salary) FROM worker w2 WHERE w2.salary > w1.salary); -- 5th highest, no
LIMIT/TOP

-- employees with the same salary as someone else
SELECT * FROM worker WHERE salary IN
(SELECT salary FROM worker GROUP BY salary HAVING COUNT(*) > 1);

-- second highest salary
SELECT MAX(salary) FROM worker WHERE salary < (SELECT MAX(salary) FROM worker);

-- departments with fewer than 5 people
SELECT department FROM worker GROUP BY department HAVING COUNT(*) < 5;

-- department headcount
SELECT department, COUNT(*) FROM worker GROUP BY department;

-- last row / first row
SELECT * FROM worker ORDER BY worker_id DESC FETCH FIRST 1 ROW ONLY;  -- last
SELECT * FROM worker ORDER BY worker_id ASC FETCH FIRST 1 ROW ONLY;   -- first
SELECT * FROM worker ORDER BY worker_id DESC FETCH FIRST 5 ROWS ONLY;  -- last 5

-- highest earner per department
SELECT * FROM worker w WHERE salary = (SELECT MAX(salary) FROM worker w2 WHERE w2.department=w.department);

-- top 3 / bottom 3 salaries
SELECT DISTINCT salary FROM worker ORDER BY salary DESC FETCH FIRST 3 ROWS ONLY;
SELECT DISTINCT salary FROM worker ORDER BY salary ASC  FETCH FIRST 3 ROWS ONLY;

-- department-wise total salary
SELECT department, SUM(salary) FROM worker GROUP BY department;

-- workers earning the max salary overall
SELECT * FROM worker WHERE salary = (SELECT MAX(salary) FROM worker);

```

A5. Date/Time functions (Oracle syntax; MySQL notes inline)

```
-- first day of month, N months before current month
SELECT ADD_MONTHS(TRUNC(SYSDATE,'MM'), -3) FROM dual;
-- last day of that month
SELECT LAST_DAY(ADD_MONTHS(TRUNC(SYSDATE,'MM'), -3)) FROM dual;

-- distinct weekday (Monday) hiredates
SELECT DISTINCT hiredate FROM emp WHERE TO_CHAR(hiredate,'DY')='MON';

SELECT TRUNC(SYSDATE,'YEAR') FROM dual; -- first day of year
SELECT ADD_MONTHS(TRUNC(SYSDATE,'YEAR'),12)-1 FROM dual; -- last day of year
SELECT TRUNC(MONTHS_BETWEEN(SYSDATE, DATE'2000-05-15')/12) AS age FROM dual;

SELECT TO_CHAR(SYSDATE,'Month DD, YYYY') FROM dual; -- "September 04, 2014"
SELECT TO_CHAR(SYSDATE,'FMDay Month YYYY') FROM dual; -- "Thursday September 2014"
SELECT TO_CHAR(SYSDATE,'YYYY') FROM dual; -- extract year
SELECT ename, hiredate FROM emp WHERE hiredate BETWEEN DATE'1987-06-01' AND DATE'1987-07-30';
SELECT TO_CHAR(SYSDATE,'FMDay DDth Month YYYY HH24:MI:SS') FROM dual;
SELECT TO_CHAR(SYSDATE,'DD/MM/YYYY') FROM dual;
SELECT TO_CHAR(SYSDATE,'HH:MI AM Mon DD, YYYY') FROM dual;

SELECT * FROM emp WHERE TO_CHAR(hiredate,'MM')='06'; -- joined in June
SELECT TO_CHAR(hiredate,'YYYY') yr, COUNT(*) FROM emp GROUP BY TO_CHAR(hiredate,'YYYY') HAVING COUNT(*)>10;
SELECT ename FROM emp WHERE TO_CHAR(hiredate,'YYYY')='1987';
SELECT ename FROM emp WHERE MONTHS_BETWEEN(SYSDATE,hiredate)/12 > 5;
SELECT deptno, TO_CHAR(hiredate,'YYYY') yr, COUNT(*) FROM emp GROUP BY deptno, TO_CHAR(hiredate,'YYYY');

-- MySQL equivalents: DATE_FORMAT(NOW(),'%M %d, %Y'), DATE_ADD, LAST_DAY(), TIMESTAMPDIFF(YEAR, hiredate,
NOW())
```

A6. GROUP BY / HAVING / aggregate functions

```
SELECT MAX(sal) "Maximum", MIN(sal) "Minimum", ROUND(SUM(sal)) "Total", ROUND(AVG(sal)) "Average"
FROM emp GROUP BY deptno;

SELECT deptno, COUNT(*) "Total Number of Managers" FROM emp WHERE job='MANAGER' GROUP BY deptno;

SELECT deptno, SUM(sal) FROM emp WHERE job <> 'MANAGER' GROUP BY deptno HAVING SUM(sal) > 20000;

SELECT job, MAX(sal), MIN(sal) FROM emp GROUP BY job;

SELECT COUNT(*) FROM emp WHERE comm IS NULL;

SELECT deptno, job, MAX(sal), AVG(sal) FROM emp GROUP BY deptno, job;

SELECT deptno, MAX(sal) FROM emp WHERE deptno > 15 GROUP BY deptno ORDER BY deptno;

SELECT deptno, SUM(sal) FROM emp GROUP BY deptno HAVING SUM(sal) > 3000;

SELECT deptno FROM emp GROUP BY deptno HAVING COUNT(*) >= 5;

SELECT job, COUNT(*) FROM emp WHERE sal > 2000 GROUP BY job;

SELECT LOWER(ename), LOWER(job) FROM emp;
```

A7. Multi-condition WHERE, CONCAT, padding, salary formulas

```
SELECT dname, ename, sal, comm, sal+NVL(comm,0)+sal*0.10 AS total_salary FROM emp e, dept d WHERE
e.deptno=d.deptno AND e.deptno IN (20,30,40);

SELECT ename "Name", job "Job" FROM emp WHERE hiredate < DATE'1986-01-01' AND sal BETWEEN 1200 AND 2500;

SELECT empno, ename, deptno FROM emp WHERE mgr = 7698;

SELECT ename, job, sal FROM emp WHERE deptno IN (10,30);

SELECT ename || ', ' || deptno AS "Emp info" FROM emp;
```

```

SELECT * FROM emp WHERE mgr IS NULL;

SELECT ename, deptno, hiredate FROM emp WHERE hiredate BETWEEN DATE'1981-01-01' AND DATE'1983-03-31' ORDER BY
hiredate;

SELECT * FROM emp WHERE job LIKE '%AGE%';

-- generate email: 2-5 chars of ename + 2-4 chars of job + domain
SELECT ename, SUBSTR(ename,2,4) || SUBSTR(job,2,3) || '@mycompany.com' AS email FROM emp;

-- RPAD for fixed length with trailing spaces (LPAD for leading)
SELECT RPAD(ename,15) AS name, job FROM emp;

SELECT mgr, AVG(sal), MAX(sal), MIN(sal) FROM emp GROUP BY mgr;

SELECT deptno, SUM(sal+NVL(comm,0)) total, AVG(sal+NVL(comm,0)) avg
FROM emp WHERE sal > 2000 AND deptno IN (10,20) GROUP BY deptno;

SELECT * FROM emp WHERE TO_CHAR(hiredate,'MON')='AUG' AND sal BETWEEN 1500 AND 2500;
SELECT * FROM emp WHERE TO_CHAR(hiredate,'MON') IN ('AUG','MAY','DEC');
SELECT ename, TO_CHAR(hiredate,'DD/MM/YY') FROM emp WHERE job='CLERK' AND comm IS NOT NULL;

SELECT SUBSTR(ename,3,3)||SUBSTR(job,-2) AS empcode, empno, ename, job FROM emp;

SELECT ename, '$'||TO_CHAR(NVL(comm,0),'FM999,999') AS comm FROM emp WHERE ename LIKE 'A%N';

SELECT empno, ename, sal, comm,
CASE WHEN comm >= 600 THEN 'Excellent Keep it up'
      WHEN comm < 600 THEN 'Good'
      ELSE 'Need improvement' END AS remark
FROM emp;

SELECT empno, ename, deptno,
CASE deptno WHEN 10 THEN 'Hr' WHEN 20 THEN 'Admin' WHEN 30 THEN 'Accounts' ELSE 'Purchase' END AS dept_name
FROM emp;

```

A8. Correlated / self comparisons (Assignments 6 & 7, "compare to X's salary")

The universal pattern: put the named person's value in a **scalar subquery**, then compare.

```

-- salary greater than SMITH's
SELECT * FROM emp WHERE sal > (SELECT sal FROM emp WHERE ename='SMITH');

-- working in SMITH's department
SELECT * FROM emp WHERE deptno = (SELECT deptno FROM emp WHERE ename='SMITH');

-- salary between RAJAN's and REVATI's
SELECT * FROM emp
WHERE sal < (SELECT sal FROM emp WHERE ename='RAJAN')
AND sal > (SELECT sal FROM emp WHERE ename='REVATI');

-- delete all employees in ALAN's department
DELETE FROM emp WHERE deptno = (SELECT deptno FROM emp WHERE ename='ALAN');

-- set ALAN's salary = MILLER's salary
UPDATE emp SET sal = (SELECT sal FROM emp WHERE ename='MILLER') WHERE ename='ALAN';

-- everyone in WARD's dept gets MILLER's salary
UPDATE emp SET sal = (SELECT sal FROM emp WHERE ename='MILLER')
WHERE deptno = (SELECT deptno FROM emp WHERE ename='WARD');

-- greater than EITHER Smith's or Alan's salary
SELECT * FROM emp WHERE sal > ANY (SELECT sal FROM emp WHERE ename IN ('SMITH','ALAN'));

-- above dept-10 average
SELECT * FROM emp WHERE sal > (SELECT AVG(sal) FROM emp WHERE deptno=10);

-- above own department's average (correlated subquery)
SELECT * FROM emp e WHERE sal > (SELECT AVG(sal) FROM emp WHERE deptno = e.deptno);

-- less than own manager's salary (self join)
SELECT e.* FROM emp e JOIN emp m ON e.mgr = m.empno WHERE e.sal < m.sal;

```

```

-- above average salary for their own job
SELECT * FROM emp e WHERE sal > (SELECT AVG(sal) FROM emp WHERE job = e.job);

-- purchase department employees (join on dept name)
SELECT e.* FROM emp e JOIN dept d ON e.deptno=d.deptno WHERE d.dname='PURCHASE';

-- name + dept name
SELECT ename, dname FROM emp e JOIN dept d ON e.deptno=d.deptno;

-- name + dept name + salary grade
SELECT ename, dname, grade FROM emp e
JOIN dept d ON e.deptno=d.deptno
JOIN salgrade s ON e.sal BETWEEN s.losal AND s.hisal;

-- employee, manager number, manager name (self join)
SELECT e.empno, e.ename, e.mgr, m.ename AS manager_name FROM emp e LEFT JOIN emp m ON e.mgr=m.empno;

```

A9. Category–Product–Salesman style joins & outer joins (Assignment 7 & SLA-1)

```

-- product, category, salesman who sold it
SELECT p.pname, c.cname, s.sname
FROM product p JOIN category c ON p.cid=c.cid
JOIN salesman_product sp ON p.pid=sp.pid
JOIN salesman s ON sp.sid=s.sid;

-- product + salesman only for salesmen in Pune
SELECT p.pname, s.sname FROM product p
JOIN salesman_product sp ON p.pid=sp.pid
JOIN salesman s ON sp.sid=s.sid
WHERE s.city='Pune';

-- product + category name
SELECT p.pname, c.cname FROM product p JOIN category c ON p.cid=c.cid;

-- courses with no room assigned -> LEFT JOIN + IS NULL
SELECT c.* FROM courses c LEFT JOIN room r ON c.rid=r.roomid WHERE r.roomid IS NULL;
-- rooms not used by any course -> the mirror RIGHT/LEFT JOIN
SELECT r.* FROM room r LEFT JOIN courses c ON r.roomid=c.rid WHERE c.rid IS NULL;

-- faculty not teaching any course
SELECT f.* FROM faculty f LEFT JOIN courses c ON f.fid=c.fid WHERE c.fid IS NULL;

-- rooms allocated OR not (i.e., all rooms with course info if any) -> plain LEFT JOIN, no filter
SELECT r.*, c.cname FROM room r LEFT JOIN courses c ON r.roomid=c.rid;

-- courses + faculty with a given special skill
SELECT c.cname, f.fname FROM courses c JOIN faculty f ON c.fid=f.fid
WHERE f.skill1='database' OR f.skill2='database';

-- full "timetable": course, faculty, room together
SELECT c.cname, f.fname, r.rname FROM courses c
LEFT JOIN faculty f ON c.fid=f.fid
LEFT JOIN room r ON c.rid=r.roomid;

-- products with category = 'chips'
SELECT p.* FROM product p JOIN category c ON p.cid=c.cid WHERE c.cname='chips';

-- products sold by a named salesman
SELECT p.* FROM product p JOIN salesman s ON p.sid=s.sid WHERE s.sname='Kirti';

-- salesmen who sold nothing (anti join)
SELECT s.* FROM salesman s LEFT JOIN product p ON s.sid=p.sid WHERE p.sid IS NULL;

-- categories with no products
SELECT c.* FROM category c LEFT JOIN product p ON c.cid=p.cid WHERE p.cid IS NULL;

-- products with no category
SELECT * FROM product WHERE cid IS NULL;

-- salesmen in cities starting with P or N
SELECT * FROM salesman WHERE city LIKE 'P%' OR city LIKE 'N%';

-- SLA-1 style aggregates

```

```

SELECT SUM(sales_qty) FROM salesman_product WHERE pid=101;           -- Q23
SELECT sid, SUM(sales_qty) FROM salesman_product GROUP BY sid;      -- Q24
SELECT cid, AVG(price) FROM product GROUP BY cid;                  -- Q25
SELECT cid, COUNT(*) FROM product GROUP BY cid HAVING COUNT(*) > 2; -- Q26
SELECT sid FROM salesman_product GROUP BY sid HAVING SUM(sales_qty) > 10; -- Q27
SELECT * FROM product WHERE price > (SELECT AVG(price) FROM product); -- Q33
SELECT sname FROM salesman WHERE salary = (SELECT MAX(salary) FROM salesman); -- Q34
SELECT p.* FROM product p JOIN category c ON p.cid=c.cid WHERE c.cname='Electronics'; -- Q35 subquery style
SELECT * FROM product WHERE pid NOT IN (SELECT pid FROM salesman_product); -- Q37 not sold
SELECT pid, sid, sales_qty FROM salesman_product sp
WHERE sales_qty = (SELECT MAX(sales_qty) FROM salesman_product WHERE pid=sp.pid); -- Q38
SELECT * FROM product ORDER BY price DESC FETCH FIRST 3 ROWS ONLY; -- Q39

```

PART B — CONSTRAINTS, DDL, DML, TRANSACTIONS (Assignment 6)

B1. CREATE TABLE with constraints

```
CREATE TABLE mydept_dbda(
  deptid  NUMBER PRIMARY KEY,
  dname   VARCHAR2(20) NOT NULL UNIQUE,
  dloc    VARCHAR2(20)
);

CREATE TABLE myemployee(
  empno    NUMBER(5) PRIMARY KEY,
  fname    VARCHAR2(15) NOT NULL,
  mname    VARCHAR2(15),
  lname    VARCHAR2(15) NOT NULL,
  sal      NUMBER(9,2) CHECK (sal >= 1000),
  doj      DATE DEFAULT SYSDATE,
  passportnum VARCHAR2(15) UNIQUE,
  deptno   NUMBER CONSTRAINT fk_deptno REFERENCES mydept_dbda(deptid) ON DELETE CASCADE
);

CREATE TABLE student(sid NUMBER PRIMARY KEY, sname VARCHAR2(20));
CREATE TABLE course(cid NUMBER PRIMARY KEY, cname VARCHAR2(20));
CREATE TABLE marks(
  studid   NUMBER,
  courseid NUMBER,
  marks    NUMBER,
  CONSTRAINT pk_marks PRIMARY KEY(studid, courseid),
  CONSTRAINT fk_sid FOREIGN KEY(studid) REFERENCES student(sid) ON DELETE SET NULL,
  CONSTRAINT fk_cid FOREIGN KEY(courseid) REFERENCES course(cid)
);

-- empty clone of emp's structure
CREATE TABLE emp10 AS SELECT * FROM emp WHERE 1=2;

-- product/salesman/category with constraints (Assignment 7, Q56)
CREATE TABLE category(cid NUMBER PRIMARY KEY, cname VARCHAR2(20) UNIQUE NOT NULL, description VARCHAR2(50));
CREATE TABLE salesman(sid NUMBER PRIMARY KEY, sname VARCHAR2(20) NOT NULL, city VARCHAR2(20));
CREATE TABLE product(
  prodid  NUMBER PRIMARY KEY,
  pname   VARCHAR2(20) NOT NULL UNIQUE,
  qty     NUMBER DEFAULT 20 CHECK (qty > 0),
  price   NUMBER(8,2),
  catid   NUMBER REFERENCES category(cid),
  sid     NUMBER REFERENCES salesman(sid)
);
ALTER TABLE salesman ADD credit_limit NUMBER;  -- Q7
```

B2. ALTER TABLE (adding constraints after the fact)

```
ALTER TABLE emp      ADD CONSTRAINT pk_emp PRIMARY KEY (empno);
ALTER TABLE dept     ADD CONSTRAINT pk_dept PRIMARY KEY (deptno);
ALTER TABLE salgrade ADD CONSTRAINT pk_sal  PRIMARY KEY (grade);

ALTER TABLE emp ADD CONSTRAINT fk_emp_dept FOREIGN KEY (deptno) REFERENCES dept(deptno);

ALTER TABLE emp ADD netsal NUMBER DEFAULT 1000;
```

B3. UPDATE / conditional UPDATE

```
UPDATE emp
SET sal = sal + sal*0.15 + NVL(comm,0), job='MANAGER', mgr=7777
```

```

WHERE deptno = 10;

UPDATE emp SET job='SENIOR CLERK' WHERE ename='SMITH';

UPDATE emp SET sal = sal + sal*0.15 WHERE comm IS NOT NULL;

```

B4. Views (Assignment 7 PDF & 8)

```

CREATE VIEW find_discount AS
SELECT c.cname, v.vname, v.price, cv.buy_price, v.price - cv.buy_price AS discount
FROM customer c
JOIN cust_vehicle cv ON c.custid = cv.custid
JOIN vehicle v      ON v.vid    = cv.vid;

SELECT * FROM find_discount;

CREATE VIEW my_hr AS
SELECT empno, ename, job, comm FROM emp WHERE comm IS NOT NULL;

CREATE VIEW mgr30 AS
SELECT * FROM emp WHERE deptno = 30;

-- Q10: insert through a view – only works if the view is "simple" (single table,
-- no joins/group by/DISTINCT) and includes all NOT NULL columns without defaults.
-- mgr30 is simple + filters deptno=30, so this INSERT succeeds and the new
-- row is created with deptno=30 (WITH CHECK OPTION would reject non-30 rows):
INSERT INTO mgr30(empno, ename, job, deptno) VALUES (9001,'RAHUL','CLERK',30);
INSERT INTO mgr30(empno, ename, job, deptno) VALUES (9002,'NEHA','CLERK',30);
INSERT INTO mgr30(empno, ename, job, deptno) VALUES (9003,'ASIF','CLERK',30);
-- find_discount CANNOT be inserted into: it joins 3 tables -> "view not
-- insertable" error, because Oracle can't tell which base table each column
-- change belongs to.

```

B5. Index

```

CREATE INDEX idx_vehicle_price ON vehicle(price);

```

B6. COMMIT / ROLLBACK / SAVEPOINT (Assignment 7 PDF, Q12-13 — walk through carefully)

```

INSERT INTO dept VALUES (50,'IT','PUNE');
INSERT INTO dept VALUES (60,'QA','PUNE');
INSERT INTO dept VALUES (70,'R&D','PUNE');
SELECT * FROM dept;          -- shows all rows including uncommitted ones (same session)

ROLLBACK;                   -- undoes everything since last COMMIT
SELECT * FROM dept;         -- the 3 new rows are GONE

-- Q13 step by step:
INSERT INTO emp(empno,ename) VALUES (100,'A');
INSERT INTO emp(empno,ename) VALUES (101,'B');
INSERT INTO emp(empno,ename) VALUES (102,'C');
SAVEPOINT A;
INSERT INTO emp(empno,ename) VALUES (103,'D');
INSERT INTO emp(empno,ename) VALUES (104,'E');
INSERT INTO emp(empno,ename) VALUES (105,'F');
SAVEPOINT B;
DELETE FROM emp WHERE empno=100;
DELETE FROM emp WHERE empno=104;
-- at this point table has: 101,102,103,105 (100 and 104 deleted)

ROLLBACK TO B;
-- undoes only the two DELETes (they came after savepoint B)
-- table now has: 100,101,102,103,104,105 <- deletes undone, all 6 inserts still there

ROLLBACK TO A;
-- undoes everything after savepoint A: the 3 inserts (103,104,105) AND the deletes

```

```
-- table now has only: 100,101,102

COMMIT;
-- makes 100,101,102 permanent. No savepoint or ROLLBACK can undo it anymore.
-- table now has: 100,101,102 (permanently)

-- ROLLBACK after COMMIT does nothing to this transaction – data is already saved.
```

Key rule: `ROLLBACK TO <savepoint>` undoes everything done *after* that savepoint but keeps the savepoint itself active; `COMMIT` wipes out all savepoints and makes changes permanent — nothing can be rolled back after that.

PART C — PL/SQL: PROCEDURES, FUNCTIONS, TRIGGERS, LOOPS (Assignment 8)

C1. Pattern printing with loops

```
-- a. simple star triangle
BEGIN
  FOR i IN 1..4 LOOP
    FOR j IN 1..i LOOP
      DBMS_OUTPUT.PUT('*');
    END LOOP;
    DBMS_OUTPUT.NEW_LINE;
  END LOOP;
END;
/

-- d. incrementing numbers
BEGIN
  FOR i IN 1..5 LOOP
    FOR j IN 1..i LOOP
      DBMS_OUTPUT.PUT(j || ' ');
    END LOOP;
    DBMS_OUTPUT.NEW_LINE;
  END LOOP;
END;
/
```

C2. Basic procedure (insert / delete / parameterized select)

```
CREATE OR REPLACE PROCEDURE insert_rec(
  peno emp.empno%TYPE, pnm emp.ename%TYPE, psal emp.sal%TYPE,
  pjob emp.job%TYPE, phiredate emp.hiredate%TYPE)
IS
BEGIN
  INSERT INTO emp(empno,ename,sal,job,hiredate) VALUES(peno,pnm,psal,pjob,phiredate);
  COMMIT;
END;
/

EXEC insert_rec(9001,'RAVI',5000,'CLERK',SYSDATE);

CREATE OR REPLACE PROCEDURE delete_emp(peno emp.empno%TYPE) IS
BEGIN
  DELETE FROM emp WHERE empno = peno;
  COMMIT;
END;
/

CREATE OR REPLACE PROCEDURE list_high_earners(p_sal emp.sal%TYPE) IS
BEGIN
  FOR r IN (SELECT e.empno,e.ename,e.deptno,d.dname FROM emp e
    JOIN dept d ON e.deptno=d.deptno WHERE e.sal > p_sal) LOOP
    DBMS_OUTPUT.PUT_LINE(r.empno||' '||r.ename||' '||r.deptno||' '||r.dname);
  END LOOP;
END;
/
```

C3. Procedure with OUT parameters

```
CREATE OR REPLACE PROCEDURE dept_sal_stats(
  p_deptno IN emp.deptno%TYPE,
  p_min OUT NUMBER, p_max OUT NUMBER, p_avg OUT NUMBER, p_cnt OUT NUMBER)
IS
BEGIN
```

```

SELECT MIN(sal),MAX(sal),AVG(sal),COUNT(*) INTO p_min,p_max,p_avg,p_cnt
FROM emp WHERE deptno=p_deptno;
END;
/

DECLARE
  v_min NUMBER; v_max NUMBER; v_avg NUMBER; v_cnt NUMBER;
BEGIN
  dept_sal_stats(10, v_min, v_max, v_avg, v_cnt);
  DBMS_OUTPUT.PUT_LINE('Min='||v_min||' Max='||v_max||' Avg='||v_avg||' Count='||v_cnt);
END;
/

```

C4. Procedure with CASE / status logic (compare to average of own group)

```

CREATE OR REPLACE PROCEDURE emp_status IS
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM emp;
  IF v_count = 0 THEN
    RAISE_APPLICATION_ERROR(-20001,'Emp table is empty');
  END IF;

  FOR r IN (SELECT e.empno,e.ename,e.job,e.sal,e.deptno,
    (SELECT AVG(sal) FROM emp WHERE deptno=e.deptno) avgsal
    FROM emp e) LOOP
    DBMS_OUTPUT.PUT_LINE(r.empno||' '||r.ename||' '||r.job||' '||r.sal||' '||
      CASE WHEN r.sal > r.avgsal THEN 'Greater'
        WHEN r.sal < r.avgsal THEN 'Lesser'
        ELSE 'Equal' END ||' '||r.deptno);
  END LOOP;
END;
/

```

C5. Salary revision by experience (IF/ELSIF ladder)

```

CREATE OR REPLACE PROCEDURE revise_sal IS
  v_exp NUMBER;
BEGIN
  FOR r IN (SELECT empno,hiredate,sal FROM emp) LOOP
    v_exp := MONTHS_BETWEEN(SYSDATE,r.hiredate)/12;
    IF v_exp <= 35 THEN
      NULL; -- no update
    ELSIF v_exp > 35 AND v_exp <= 38 THEN
      UPDATE emp SET sal = sal + sal*0.20 WHERE empno=r.empno;
    ELSE
      UPDATE emp SET sal = sal + sal*0.25 WHERE empno=r.empno;
    END IF;
  END LOOP;
  COMMIT;
END;
/

```

C6. Function + procedure combo (experience → allowance)

```

CREATE OR REPLACE FUNCTION get_experience(p_hiredate DATE) RETURN NUMBER IS
BEGIN
  RETURN TRUNC(MONTHS_BETWEEN(SYSDATE,p_hiredate)/12);
END;
/

CREATE TABLE emp_allowance(
  empno NUMBER, ename VARCHAR2(20), hiredate DATE,
  experience NUMBER, allowance NUMBER(9,2)
);

CREATE OR REPLACE PROCEDURE calc_allowance IS
  v_exp NUMBER;

```

```

BEGIN
  FOR r IN (SELECT empno,ename,hiredate FROM emp) LOOP
    v_exp := get_experience(r.hiredate);
    INSERT INTO emp_allowance VALUES (r.empno,r.ename,r.hiredate,v_exp,v_exp*3000);
  END LOOP;
  COMMIT;
END;
/

```

C7. Cost-to-company function

```

CREATE OR REPLACE FUNCTION cost_to_company(p_sal NUMBER, p_hiredate DATE) RETURN NUMBER IS
  v_service NUMBER; v_special NUMBER := 0; v_da NUMBER; v_hra NUMBER; v_ta NUMBER;
BEGIN
  v_service := MONTHS_BETWEEN(SYSDATE,p_hiredate)/12;
  v_da := p_sal*0.15; v_hra := p_sal*0.20; v_ta := p_sal*0.08;
  IF v_service < 1 THEN v_special := 0;
  ELSIF v_service < 2 THEN v_special := p_sal*0.10;
  ELSIF v_service < 4 THEN v_special := p_sal*0.20;
  ELSE v_special := p_sal*0.30;
  END IF;
  RETURN p_sal + v_da + v_hra + v_ta + v_special;
END;
/

SELECT empno, ename, sal, cost_to_company(sal,hiredate) AS ctc FROM emp;

```

C8. Triggers

```

-- Q1: backup old salary before update
CREATE TABLE emp_back(empno NUMBER, ename VARCHAR2(20), oldsal NUMBER(9,2), newsal NUMBER(9,2));

CREATE OR REPLACE TRIGGER trg_emp_backup
BEFORE UPDATE OF sal ON emp
FOR EACH ROW
BEGIN
  INSERT INTO emp_back VALUES (:OLD.empno, :OLD.ename, :OLD.sal, :NEW.sal);
END;
/

-- Q2: audit insert/delete
CREATE TABLE empaudit(empno NUMBER, ename VARCHAR2(20), username VARCHAR2(20), chdate DATE, action
VARCHAR2(20));

CREATE OR REPLACE TRIGGER trg_emp_audit
AFTER INSERT OR DELETE ON emp
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO empaudit VALUES (:NEW.empno,:NEW.ename,USER,SYSDATE,'INSERT');
  ELSIF DELETING THEN
    INSERT INTO empaudit VALUES (:OLD.empno,:OLD.ename,USER,SYSDATE,'DELETE');
  END IF;
END;
/

-- Q3: vehicle price change history
CREATE TABLE vehicle_history(
  vno NUMBER, vname VARCHAR2(20), oldprice NUMBER(9,2),
  newprice NUMBER(9,2), chdate DATE, username VARCHAR2(20)
);

CREATE OR REPLACE TRIGGER trg_vehicle_price
BEFORE UPDATE OF price ON vehicle
FOR EACH ROW
BEGIN
  INSERT INTO vehicle_history VALUES (:OLD.vid,:OLD.vname,:OLD.price,:NEW.price,SYSDATE,USER);
END;
/

```


PART D — CASSANDRA CQL (Assignment 9)

D1. Keyspace & basic table with collections

```
CREATE KEYSPACE college WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE college;

CREATE TABLE student(
    sid      int,
    courseid int,
    sname    text,
    age      int,
    mobile    text,
    email     text,
    hobbies   set<text>,
    project   map<text,int>,
    skill     list<text>,
    marks     tuple<text,int,text,int>,
    PRIMARY KEY (courseid, sid)
);

INSERT INTO student(sid,courseid,sname,age,mobile,email,hobbies,project,skill,marks)
VALUES (1,101,'Amit',22,'9999999999','amit@x.com',{'reading','music'},{'proj1':80},['python','java'],('unit1',
80,'unit2',85));
-- repeat with sid 2,3,4,5 for the other courseids

SELECT * FROM student;

UPDATE student SET marks = ('unit1',92,'unit2',90) WHERE courseid=101 AND sid=2;

DELETE FROM student WHERE sid=2 AND courseid=101;

UPDATE student SET email = null WHERE sid=3 AND courseid=102;

UPDATE student SET skill = skill + ['java','.net'] WHERE sid=1 AND courseid=102;
UPDATE student SET skill = skill - ['python'] WHERE sid=1 AND courseid=102;
```

D2. Course table with map + set

```
CREATE TABLE course(
    cid      int PRIMARY KEY,
    cname    text,
    subject  set<text>,
    skills   set<text>,
    marks    map<text,int>
);

UPDATE course SET subject = subject + {'Aptitude'} WHERE cid = /* DBDA's cid */ 1;
UPDATE course SET marks['Aptitude'] = 90 WHERE cid = 1;
UPDATE course SET skills = skills - {'python'} WHERE cid = /* DAC's cid */ 2;
UPDATE course SET marks['Java'] = 45 WHERE cid = /* course with Java */ 1;
```

D3. E-commerce: product table (plain columns)

```
CREATE KEYSPACE ecommerce WITH replication = {'class':'SimpleStrategy','replication_factor':1};
USE ecommerce;

CREATE TABLE product(
    product_id int PRIMARY KEY,
    name text, category text, price double, stock int, created_at timestamp
);

INSERT INTO product(product_id,name,category,price,stock) VALUES (101,'Laptop','Electronics',65000,10);
INSERT INTO product(product_id,name,category,price,stock) VALUES (102,'Mobile','Electronics',20000,20);
INSERT INTO product(product_id,name,category,price,stock) VALUES (103,'Headphones','Electronics',2000,50);
```

```

INSERT INTO product(product_id,name,category,price,stock) VALUES (104,'Office Chair','Furniture',8000,15);
INSERT INTO product(product_id,name,category,price,stock) VALUES (105,'Study Table','Furniture',12000,5);

SELECT * FROM product;
SELECT name, price FROM product;
SELECT * FROM product WHERE product_id = 103;
SELECT * FROM product WHERE category = 'Electronics' ALLOW FILTERING;
UPDATE product SET price = 18000 WHERE product_id = 102;
UPDATE product SET stock = 8 WHERE product_id = 105;
UPDATE product SET category = 'Office Furniture' WHERE product_id = 104;
DELETE FROM product WHERE product_id = 103;
DELETE stock FROM product WHERE product_id = 101;
DELETE FROM product WHERE product_id = 104; -- Cassandra deletes by key, not by non-key column value

```

D4. Customer table with SET / LIST / MAP columns

```

CREATE TABLE customer(
  customer_id int PRIMARY KEY,
  name text,
  phones set<text>,
  orders list<int>,
  address map<text,text>
);

INSERT INTO customer(customer_id,name,phones,orders,address)
VALUES (1,'Priya',{'9876543210','9123456780'},[101,102,103],{'city':'Pune','state':'Maharashtra'});
INSERT INTO customer(customer_id,name,phones,orders,address)
VALUES (2,'Amit',{'9998887776'},[104,105],{'city':'Mumbai','state':'Maharashtra'});
INSERT INTO customer(customer_id,name,phones,orders,address)
VALUES (3,'Neha',{'8887776665','7776665554'},[101],{'city':'Delhi','state':'Delhi'});

SELECT * FROM customer;
SELECT phones FROM customer WHERE customer_id = 1;
SELECT orders FROM customer WHERE customer_id = 2;
SELECT address['city'] FROM customer WHERE customer_id = 1;

UPDATE customer SET phones = phones + {'9999999999'} WHERE customer_id = 1;
UPDATE customer SET orders = orders + [106] WHERE customer_id = 2;
UPDATE customer SET address['city'] = 'Bangalore' WHERE customer_id = 3;
UPDATE customer SET phones = phones - {'9123456780'} WHERE customer_id = 1;
UPDATE customer SET orders = orders - [103] WHERE customer_id = 1;
DELETE address['state'] FROM customer WHERE customer_id = 2;

```

D5. User Defined Type (UDT) + frozen column

```

CREATE TYPE payment_info(method text, status text);

CREATE TABLE orders(
  order_id int PRIMARY KEY,
  product_name text,
  amount double,
  payment_details frozen<payment_info>
);

INSERT INTO orders(order_id,product_name,amount,payment_details)
VALUES (1,'Laptop',65000,{method:'Credit Card',status:'Paid'});
INSERT INTO orders(order_id,product_name,amount,payment_details)
VALUES (2,'Mobile',20000,{method:'UPI',status:'Paid'});
INSERT INTO orders(order_id,product_name,amount,payment_details)
VALUES (3,'Chair',8000,{method:'Cash',status:'Pending'});

SELECT * FROM orders;
SELECT payment_details.method FROM orders WHERE order_id = 1;
UPDATE orders SET payment_details = {method:'Cash',status:'Paid'} WHERE order_id = 3; -- frozen UDT: replace whole value
DELETE FROM orders WHERE order_id = 2;

```

Cassandra rule of thumb: `+` / `-` work on `set` / `list` / `map` columns to add/remove elements without rewriting the whole collection; a **frozen** UDT/collection must always be overwritten as a whole value.

PART E — MONGODB UPDATE OPERATORS

(mongodbassignment_update_final.pdf)

Sample document shape used throughout:

```
{
  empno: 111, ename: "Deepali Vaidya", sal: 40000, desg: "Analyst",
  dept: { deptno: 12, dname: "Hr", dloc: "Mumbai" },
  mgr: { name: "Satish", num: 222 },
  project: [ {name:"project-1", hrs:4}, {name:"project-2", hrs:4} ],
  skillset: ["perl","python","java"],
  hobbies: ["reading","biking"],
  bonus: 0
}
```

E1. Simple field update (`$set`) matching a condition

```
// 1. rename CLERK desg to A0
db.employee.updateMany({desg:"CLERK"}, {$set:{desg:"A0"}});

// 7. change manager name+number wherever manager is Satish
db.employee.updateMany({"mgr.name":"Satish"}, {$set:{"mgr.name":"Tushar","mgr.num":3333}});

// 8. raise salary for purchase dept
db.employee.updateMany({"dept.dname":"Purchase"}, {$inc:{sal:15000}});

// 11. cap up low salaries
db.employee.updateMany({sal:{$lt:10000}}, {$set:{sal:10000}});

// 21. replace a whole document
db.employee.replaceOne({ename:"Deepak"}, { /* entirely new document fields */ });

// 22. rename a value inside an array (arrayFilters lets you target the matching element)
db.employee.updateMany(
  {skillset:"python"},
  {$set:{"skillset.$elem":"python 3.8"}},
  {arrayFilters:[{elem:"python"}]}
);
```

E2. Updating one matching element of an array (positional `$`)

```
// 2. project-1 hours -> 5, only for analysts
db.employee.updateMany(
  {desg:"Analyst", "project.name":"project-1"},
  {$set:{"project.$.hrs":5}}
);

// 15. desg -> Senior Programmer if they work project-1 for 4 hrs
db.employee.updateMany(
  {project:{$elemMatch:{name:"Project-1",hrs:4}}},
  {$set:{desg:"Senior Programmer"}}
);

// 27. bump project-1 hrs to 6 only if currently < 6 (otherwise leave alone)
db.employee.updateMany(
  {"project.name":"project-1","project.hrs":{$lt:6}},
  {$set:{"project.$.hrs":6}}
);
```

E3. Adding elements to arrays (\$push)

```
// 3. add 2 projects for names starting with "Deep"
db.employee.updateMany(
  {ename:{$regex:"^Deep"}},
  {$push:{project:{$each:[{name:"project-3",hrs:2},{name:"project-4",hrs:2}]}}}
);

// 13. insert 2 new projects at index 2 for analysts or specific salaries
db.employee.updateMany(
  {$or:[{desg:"Analyst"},{sal:{$in:[30000,33000,35000]}]}},
  {$push:{project:{$each:[{name:"projX",hrs:0},{name:"projY",hrs:0}], $position:2}}}
);

// 16. add hobbies list for employees managed by Rajan or Revati
db.employee.updateMany(
  {"mgr.name":{$in:["Rajan","Revati"]}},
  {$set:{hobbies:["swimming","chess"]}}
);

// 17. add skillset for people on project-4(3hrs) or project-3(4hrs)
db.employee.updateMany(
  {$or:[
    {project:{$elemMatch:{name:"project-4",hrs:3}}},
    {project:{$elemMatch:{name:"project-3",hrs:4}}}
  ]},
  {$set:{skillset:["mongodb","aws"]}}
);

// 18. insert "blogging" at position 3 in hobbies for names starting R/P, ending j/s
db.employee.updateMany(
  {ename:{$regex:"^[RPrp].*[jsJS]$"}},
  {$push:{hobbies:{$each:["blogging"], $position:3}}}
);

// 23. append 2 skills at the END for Pune location
db.employee.updateMany(
  {"dept.dloc":"Pune"},
  {$push:{skillset:{$each:["MongoDB","Perl"]}}}
);

// 26. append 2 new projects at end if skillset has perl or python
db.employee.updateMany(
  {skillset:{$in:["Perl","python"]}},
  {$push:{project:{$each:[{name:"project-5",hrs:0},{name:"project-6",hrs:0}]}}}
);
```

E4. Removing elements from arrays (\$pull , \$pop)

```
// 10. delete project-2 wherever its hrs = 4
db.employee.updateMany({}, {$pull:{project:{name:"project-2",hrs:4}}});

// 14. delete last project for HR dept in Mumbai
db.employee.updateMany(
  {"dept.dname":"HR","dept.dloc":"Mumbai"},
  {$pop:{project:1}} // 1 = remove last, -1 = remove first
);

// 20. remove all employees working on project-1 (removes the whole document!)
db.employee.deleteMany({"project.name":"project-1"});

// 24. delete first hobby for anyone on project-1 or project-2
db.employee.updateMany(
  {"project.name":{$in:["project-1","project-2"]}},
  {$pop:{hobbies:-1}}
);

// 25. delete last hobby if project at ARRAY INDEX 1 (2nd position) has hrs=4
db.employee.updateMany(
  {"project.1.hrs":4},
```

```
{ $pop: { hobbies: 1 } }
});
```

E5. Conditional numeric increments (`$inc` with `$or`)

```
// 4/5/6: tiered bonus by salary bracket
db.employee.updateMany({sal:{>:50000}}, {$inc:{bonus:2000}});
db.employee.updateMany({sal:{<:50000,>:30000}}, {$inc:{bonus:1500}});
db.employee.updateMany({sal:{<:30000}}, {$inc:{bonus:1000}});

// 9. reduce project-2 hours by 2
db.employee.updateMany({"project.name":"project-2"}, {$inc:{"project.$.hrs":-2}});

// 12. bonus+500 if bonus<2000 OR sal<20000 OR sales dept
db.employee.updateMany(
  {$or:[{bonus:{<:2000}}, {sal:{<:20000}}, {"dept.dname":"Sales"}]},
  {$inc:{bonus:500}}
);

// 19. raise sal for project-1/2/3 workers; separately adjust Mumbai-located staff
db.employee.updateMany({"project.name":{"in":["project-1","project-2","project-3"]}}, {$inc:{sal:10000}});
db.employee.updateMany({"dept.dloc":"Mumbai"}, {$inc:{bonus:-1000, sal:1000}});
```

MongoDB cheat-sheet: `$set` overwrite one field · `$inc` add/subtract a number · `$push` / `$pop` / `$pull` add/remove-end/remove-matching on arrays · `arrayFilters` + `[$elem]` or the positional `$` to target one array element · `$elemMatch` to match an array element on multiple conditions at once · `$or` / `$in` / `$regex` to combine or pattern-match conditions.

Quick lookup: "if the question says... use..."

Question phrase	Technique
"greater/less than X's salary"	scalar subquery: <code>(SELECT sal FROM emp WHERE ename='X')</code>
"average of own department/job"	correlated subquery: <code>WHERE sal > (SELECT AVG(sal) FROM emp WHERE deptno=e.deptno)</code>
"not assigned / no matching row"	<code>LEFT JOIN ... WHERE right.key IS NULL</code>
"for each department/job"	<code>GROUP BY</code>
"only groups where count/sum > N"	<code>HAVING</code> (never <code>WHERE</code> for aggregates)
"nth highest"	subquery counting distinct greater values, or <code>ORDER BY ... FETCH FIRST n ROWS ONLY</code>
"starts with / ends with / contains"	<code>LIKE 'X%' / '%X' / '%X%'</code>
"exact position of a letter"	count underscores in <code>LIKE</code> pattern
a value inside an array/document (Mongo)	<code>\$elemMatch</code> , dot notation, positional <code>\$</code>
add/remove one element (Mongo array)	<code>\$push</code> / <code>\$pull</code> / <code>\$pop</code>
add/remove one element (Cassandra collection)	<code>col = col + {...}</code> / <code>col = col - {...}</code>